SCHULUNG

Software gemeinsam entwickeln

Mit Fachwissen und KI-Agenten: wie Domänenexperten und IT-Experten zusammen sichere, wartbare Software bauen

Simon Lauger, Lizenz CC BY-NC-SA 4.0

Worum es geht

Zusammenarbeit von Domänenexperten und IT-Experten

DOMÄNENEXPERTEN

Entwickeln mit KI-Agenten mit

Fachleute aus der Praxis tragen die Entwicklung mit, statt nur Tickets zu eröffnen.

IT-EXPERTEN

Sichern ab

Architektur und Sicherheit: hinterfragen, beraten und per Review freigeben.

GEMEINSAM

Nichts ohne Review

Jede Änderung läuft über Pull Request, automatische Prüfungen und ein menschliches Review.

Domänenexperten bekommen mehr Wissen, mehr Werkzeuge und mehr Verantwortung. IT-Experten bleiben verantwortlich für Datenmodell, Architektur, Sicherheit, Datenschutz und Wartung.

Ablauf der Schulung

TEIL 1

Warum

Bedarf, Kosten, zwei Lücken, geteilte Verantwortung, das Beispiel

TEIL 2

Ablauf

Rollen, von der Idee bis zum Release, Konzept, Recherche, Datenmodell

TEIL 3

Bauen

Coding-Agent, Review, Testen, Umgebungen, Kosten, Modelle, Datenschutz

TEIL 4

Sicher betreiben

Sicherheit, Prototyp und Betrieb, Updates, CVE, SBOM, Lebenszyklus

TEIL 5

Miteinander

Lizenzen und Forks, gute Fragen, Reviews, Verhaltenskodex

TEIL 6

Loslegen

Mindestwissen, Lernbegleiter, erste Schritte

TEIL 1

Warum

Warum Fachleute heute mitbauen sollen, und warum das nur gemeinsam geht

IT-Experten allein können den Bedarf an Software nicht mehr decken.

Fachleute haben die richtig guten Ideen, weil sie die Abläufe jeden Tag erleben.

Lizenzkosten, die sich ein Verein nicht leisten kann

EINNAHMEN

Beiträge, Spenden, kleine Zuschüsse

Davon leben Vereine, Feuerwehren und andere Ehrenamtsorganisationen.

FACHSOFTWARE

Mehrere hundert bis tausende Euro pro Jahr

Meist pro Benutzer oder pro Modul.

FOLGE

Das Geld fehlt woanders

Schon wenige hundert Euro sind für viele Vereine ein Problem.

Eigene, gemeinsam gepflegte Software kann diese Kosten sparen, und als Open Source auch für andere Vereine.

Ehrlich gerechnet: selbst gebaut ist nicht kostenlos

Betrieb

Server, Datenbank, Sicherungen, Zertifikate

Tägliche Wartung

Updates und Sicherheitskorrekturen, sonst wird sie angreifbar

Verantwortliche

Wer kümmert sich in drei Jahren noch darum?

Daten der Mitglieder

Verantwortung bis hin zu Gesundheitsdaten

Der Weg lohnt sich nur mit einem Prozess, der Qualität, Sicherheit und Wartung von Anfang an mitdenkt.

Zwei Seiten, zwei Lücken

DOMÄNENEXPERTE

Weiß genau

wie ein Ablauf in der Praxis funktioniert, was wirklich gebraucht wird, welche Sonderfälle es gibt

Weiß oft nicht

was ein Datenmodell ist, was TLS, Passwort-Hashing, Rechteprüfung, Datenschutz und Updates bedeuten

Typisches Risiko

Ohne Plan wird "irgendwas reingehackt". Es funktioniert, bis es gehackt wird.

IT-EXPERTE

Weiß genau

wie man Abläufe in Datenmodell und Code übersetzt und Software sicher, wartbar und betreibbar macht

Weiß oft nicht

wie der Ablauf in der Praxis wirklich aussieht und was den Leuten im Alltag wichtig ist

Typisches Risiko

Sauber gebaut, löst aber das falsche Problem. Keine Zeit für jede Domäne.

Der Zwischenweg

1

Domänen-experte treibt

beschreibt, diskutiert, lässt den Agenten umsetzen, prüft fachlich

2

Architekt sichert ab

hinterfragt Konzepte, prüft jeden Pull Request, schickt die Leute wieder los

3

Eigene Sicherheits-prüfung

automatische Werkzeuge, bei kritischen Änderungen ein Mensch

4

Nichts ohne Review

Pull Request, automatische Prüfungen, KI-Review, menschliches Review

5

Entscheidungen aufschreiben

als ADR, damit später niemand raten muss, auch kein Agent

> Der übliche Weg reicht nicht mehr: Fachleute eröffnen Tickets, IT-Experten setzen um, und das meiste bleibt liegen.

Geteilte Verantwortung

Domänenexperten *was IN der Software passiert*

- Ideen und Wünsche
- Abläufe und Begriffe
- Fachliche Regeln und Sonderfälle
- Testfälle in eigenen Worten
- Umsetzung mit dem Agenten
- Fachliche Prüfung und Abnahme
- Kameraden einweisen

Gemeinsam *braucht Fach- und IT-Wissen*

- Anforderungen und Scope
- Datenmodell
- Rechte und Rollen
- Datenschutz: welche Daten, wofür
- Priorisierung
- Entscheidungen festhalten (ADR)

IT-Experten *die Software SELBST*

- Architektur
- Sicherheit und Review
- Tests und CI
- Betrieb und Hosting
- Monitoring
- Backups
- Updates, CVEs, SBOM
- Releases und Rollback
- Zugänge und Geheimnisse

KI-Agent

setzt um, was Menschen beauftragen und prüfen.
Trägt keine Verantwortung.

"Der Agent hat das so gebaut" ist keine Begründung.

Wer den Auftrag gibt und den Pull Request einreicht, muss erklären können, was sich ändert und warum.

Das Beispiel: Feuerwehr Musterstadt

Erfunden, aber realistisch. Die Software heißt "Wachbuch".

70
aktive Mitglieder in zwei Löschzügen

25
davon Atemschutz-geräteträger

6
Fahrzeuge, darunter HLF 20 und Drehleiter

200
Mitglieder im Förderverein

Das Wachbuch verwaltet Mitglieder, Dienstzeiten, Fahrzeuge und Geräteprüfungen. Bisher läuft das auf Papier, in Tabellen und in Chatgruppen.

Wer mitmacht

KOMMANDANT

Prioritäten, Pflichten

entscheidet, was zuerst gebaut wird und wer was sehen darf

KARL

Atemschutz-gerätewart

kennt Prüffristen und Einsatzbereitschaft, Domänenexperte

GERÄTEWART

Fahrzeuge und Geräte

Prüfintervalle, Fahrzeugcheck am Handy in der Halle

JUGENDWART

Jugendfeuerwehr

Einverständnisse, Notfallkontakte, Daten Minderjähriger

KASSIERERIN

Förderverein

Beiträge, Lastschrift, Bankdaten schützen

JONAS

Junger Feuerwehrmann

schnell mit KI-Werkzeugen, hält "läuft bei mir" für "fertig"

PETER

Softwarearchitekt, extern

Maintainer mehrerer Vereinsprojekte, wenig Zeit

SABINE

IT-Sicherheit, optional

Sicherheit und Datenschutz, sonst übernimmt Peter

TEIL 2

Ablauf

Wer was macht, und welchen Weg eine Idee bis in die Software nimmt

Die Rollen

DOMÄNENEXPERTE

Beschreibt, baut mit dem Agenten, prüft fachlich

Darf nie: etwas ohne Review in die Software bringen oder echte Daten an KI-Werkzeuge geben

CODING-AGENT

Plant, schreibt Code und Tests, fragt nach

Darf nie: entscheiden, was gebaut wird, oder Prüfungen abschalten, damit es durchläuft

ARCHITEKT UND MAINTAINER

Prüft Konzepte und Pull Requests, entscheidet

Darf nie: freigeben, was er nicht verstanden hat, oder Review überspringen, weil es eilt

SECURITY (OPTIONAL)

Prüft Anmeldung, Rechte, persönliche Daten

Darf nie: Lücken "später" beheben lassen, wenn sie jetzt schaden können

KI-REVIEW-BOT

Meldet früh Auffälligkeiten

Darf nie: allein über eine Freigabe entscheiden. Kommentare sind Hinweise.

CI

Prüft jede Änderung, meldet rot oder grün

Darf nie: abgeschaltet oder umgangen werden

Von der Idee bis zum Release

1
Idee per Sprache
im Gespräch durchdenken

2
Konzept
Review durch den Architekten

3
Issue
Aufgabe mit Abnahme-kriterien

4
Prompt
Auftrag an den Agenten

5
Agent
plant, Freigabe, setzt um

6
Eigene Prüfung
ausprobieren, hinterfragen

7
Pull Request
Änderungs-vorschlag

8
Automatisch + KI-Review
CI prüft, Bot kommentiert

9
Review IT-Experten
Architekt, ggf. Security

10
Merge
in die Hauptversion

11
Test
Testumgebung, erfundene Daten

12
Release
kommt bei der Wehr an

> Zurückgehen ist normal und kein Scheitern. Je später ein Fehler auffällt, desto teurer wird er.

Nichts landet ohne Review in der Software.

Egal wie klein die Änderung ist, egal wie sicher sich der Agent ist, egal wie eilig es ist.

Ideen im Gespräch entwickeln

Karl, auf dem Parkplatz nach der Übung

Nach einem Einsatz trage ich die Atemschutzzeiten zweimal ein: im Einsatzbericht und im Nachweis jedes Geräteträgers. Das will ich automatisch haben.

Assistent

Was ist die Wahrheit, wenn sich beide widersprechen? Der Einsatzbericht oder der Nachweis?

Karl

Eigentlich der Einsatzbericht. Da steht ja, was passiert ist.

Assistent

Dann sollte der Nachweis die Zeiten nicht kopieren, sondern auf den Bericht verweisen. Und was passiert, wenn ein Bericht storniert wird?

Sprache ist zum Denken da

GUT GEEIGNET

- laut denken, Gedanken sortieren
- Fragen stellen lassen, die man selbst übersieht
- den Assistenten widersprechen lassen
- Risiken und Sonderfälle finden

WENIGER GEEIGNET

- genaue Anforderungen festlegen
- Code schreiben oder prüfen
- Tabellen, Zahlen, Feldnamen
- endgültige Entscheidungen

> Am Anfang sagen: "Stell mir Fragen und sag mir ehrlich, wo die Idee Schwächen hat." Keine echten Namen oder Diagnosen.

Erst Konzept, dann Code

FEHLER IM PLAN

Radiergummi

fünf Minuten

NACH DEM FUNDAMENT

Bagger

eine Woche

NACH DEM EINZUG

Umbau im laufenden Betrieb

Monate

Ein Konzept von einer Seite liest der Architekt in zehn Minuten. Ein Pull Request mit 40 Dateien und falschem Ansatz kostet ihn eine Stunde und dich die ganze Arbeit.

Konzept, Issue, Prompt

KONZEPT

Was und warum, nicht wie

- Problem, Ziel, Nutzer, Ablauf
- Daten: Quelle der Wahrheit?
- Rechte und Datenschutz
- Grenzen und offene Fragen
- Abnahmekriterien

ISSUE

Fester Ort für die Aufgabe

- Worum es geht, Link zum Konzept
- Abnahmekriterien als Liste
- Was nicht dazugehört
- Abhängigkeiten, z. B. ein ADR

PROMPT

Auftrag an den Agenten

- Issue und AGENTS.md lesen
- erst Plan, auf OK warten
- kleine Schritte, Tests je Kriterium
- keine Prüfung abschalten
- nur Testdaten, fragen statt raten

Schauen, wie andere es lösen

ERKENNTNIS

Erfasst wird ein Trupp, nicht eine Person

Ohne Recherche hätte Karl Zeiten pro Person gebaut und später alles umbauen müssen.

GRENZE

Inspiration ja, kopieren nein

Ideen und Abläufe anschauen ist erlaubt. Code, Bilder, Texte und Handbücher sind geschützt.

VORSICHT

Der Assistent kann halluzinieren

Funktionen, Paragrafen, Zahlen und Fristen immer an der Quelle prüfen.

> Eine Stunde Recherche spart oft Tage. Manchmal ist "nicht bauen" die beste Entscheidung.

Das Datenmodell ist das Fundament

Daten überleben den Code. Fehler darin sind teuer und manchmal gefährlich.

IDENTITÄT

E-Mail als Login-Nummer

Familie mit gemeinsamer Adresse: nur einer kann sich anmelden. Neue Adresse: neue Person, leerer Nachweis.

LÖSCHEN

Ausgetretene endgültig löschen

Teilnahmen verschwinden mit. Die Statistik für die Gemeinde stimmt nicht mehr.

MANDANTEN

Nachbarwehr in dieselbe Datenbank

Ein vergessener Filter, und sie sieht die Mitglieder aus Musterstadt.

GESUNDHEIT

Untersuchung als Feld beim Mitglied

Gesundheitsdaten landen in Exporten, Listen und Telefonlisten.

Besser: feste Nummer je Person, stornieren statt löschen, Mandant von Anfang an, sensible Daten in eigener Entität mit eigenen Rechten.

Datenmodell-Fragen erkennen und festhalten

BERÜHRT MEINE IDEE DAS FUNDAMENT?

- Muss sich die Software etwas Neues merken?
- Entsteht eine neue Beziehung?
- Ändert sich, was eindeutig ist?
- Geht es um Löschen, Stornieren, Archivieren?
- Sind sensible Daten betroffen?
- Steht dieselbe Information künftig an zwei Stellen?
- Sollen andere Organisationen mitnutzen?

ADR: ARCHITEKTURENTSCHEIDUNG

Eine Seite im Projekt

- **Kontext:** welches Problem?
- **Entscheidung:** was gilt?
- **Alternativen:** was nicht, und warum?
- **Folgen:** auch die Nachteile

In zwei Jahren steht die Antwort da, auch für den Agenten.

TEIL 3

Bauen

Mit dem Agenten arbeiten, prüfen, testen, und die richtigen Werkzeuge wählen

Mit dem Coding-Agenten arbeiten

WAS ER IST

Eine sehr schnelle, sehr belesene Aushilfe

Kennt aber das Projekt nicht, die Feuerwehr nicht und erzählt manchmal mit großer Überzeugung Unsinn.

EINSTIEG

In der Cloud, im Browser starten

Keine Installation: GitHub verbinden, Repository wählen, Auftrag geben, Diff ansehen.

WICHTIGSTE REGEL

Verstehen, was sich ändert

In drei Sätzen: welche Dateien, was Nutzer anders erleben, welche Daten betroffen sind.

Welcher Anbieter? Der, für den du schon ein Abo hast. Das Vorgehen ist wichtiger als das Werkzeug.

Den Agenten führen und hinterfragen

"Warum?"

Warum diese Datei, warum ein neues Feld? Eine ausweichende Antwort ist ein Warnsignal.

"Welche Alternativen gibt es?"

Zwei oder drei Wege mit Vor- und Nachteilen. Oft ist der einfachste der beste.

"Was könnte kaputtgehen?"

Welche bestehenden Funktionen und welche vorhandenen Daten sind betroffen?

"Wer darf das sehen oder ändern?"

Bei jeder Änderung an Daten und Ansichten nach den Rechten fragen.

"Erkläre es mir für den Architekten."

Kannst du es nicht wiedergeben, ist der Pull Request nicht fertig.

Stopp, wenn er sich im Kreis dreht

Derselbe Fehler zum dritten Mal, Tests löschen, Prüfungen abschwächen: abbrechen.

Pull Request und Review-Kette

1

CI

Tests, Lint, Typprüfung, Secret-Scan, Abhängigkeiten, Build

2

KI-Review

Kommentare zu Fehlern, unklaren Stellen, fehlenden Tests

3

Architekt

Richtiges Problem? Datenmodell? Rechte auf dem Server? Einfacher?

4

Security

Bei Anmeldung, Rechten, persönlichen Daten, neuen Paketen

Rot heißt: nicht mergen. Der KI-Reviewer gibt Hinweise, keine Freigabe. Gemergt wird erst, wenn alles grün, beantwortet und freigegeben ist.

Der Architekt darf Nein sagen

Auch wenn es funktioniert. Er verantwortet, dass die Software in fünf Jahren noch wartbar und sicher ist.

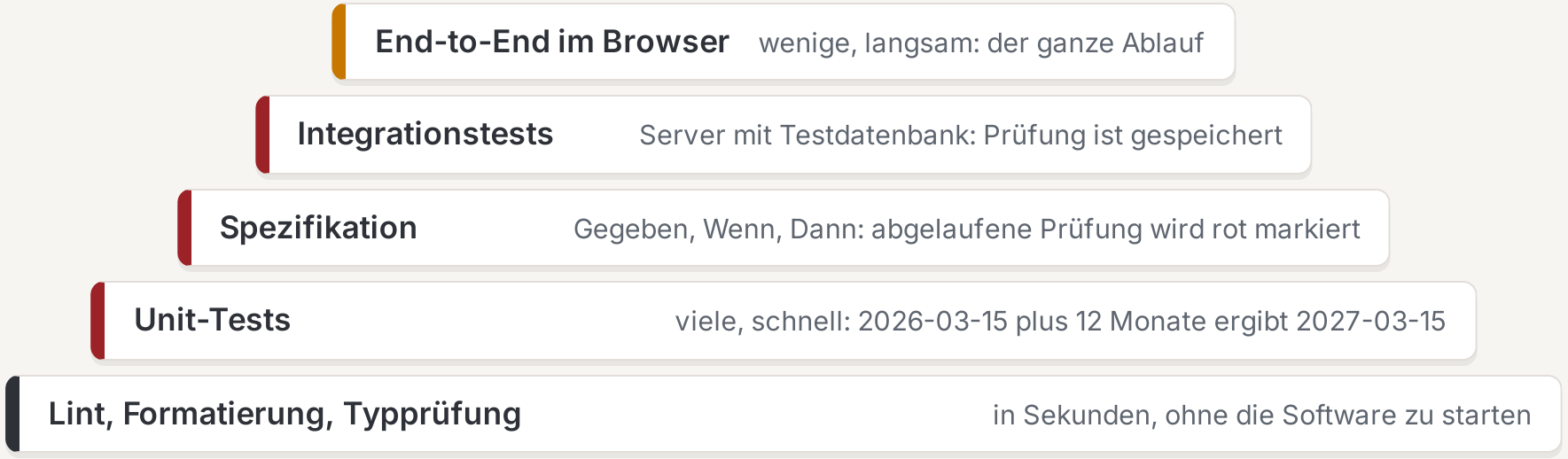
- Zu groß oder mehrere Themen vermischt
- Datenmodell so geändert, dass es später teuer wird
- Rechte nur in der Oberfläche statt auf dem Server
- Neue Abhängigkeit oder externer Dienst ohne Begründung
- Prüfungen abgeschwächt, Tests gelöscht
- Der Einreichende kann nicht erklären, was der Pull Request tut

> Abgelehnt heißt selten endgültig. Meist reicht es, zu verkleinern, aufzuteilen oder vorher im Issue die Richtung zu klären.

Testen ist der wichtigste Punkt in der KI-Entwicklung.

Ein Agent ändert in Minuten dutzende Dateien. Nur Tests zeigen, ob das Alte noch funktioniert.

Die Testpyramide



> Alle Tests laufen bei jedem Pull Request. Ein roter Test wird repariert, nicht gelöscht, übersprungen oder abgeschwächt.

Dein Beitrag: Sonderfälle in eigenen Worten

Issue: erwartetes Verhalten

```
Untersuchung gültig, mehr als 3 Monate:
grün
weniger als 3 Monate: gelb
abgelaufen: rot, nicht einsatzbereit
Sonderfälle:
noch nie untersucht: rot, nicht leer
läuft genau heute ab: gilt heute noch
Alters- und Ehrenabteilung: taucht nicht
auf
Schaltjahr: 29. Februar, Frist 36 Monate
```

1

Nachstellen

mit erfundenen Daten, nie mit Produktionsdaten

2

Roter Test

muss vor der Korrektur fehlschlagen

3

Korrigieren

bis der Test grün ist

4

Alle Tests

nichts anderes kaputt

Umgebungen: wo darf ein KI-Agent hin?

Umgebung	Daten	KI-Agent darf zugreifen?
Lokal	erfunden	**ja**
Preview pro Pull Request oder Entwickler	erfunden	**ja**
Dev/Test (gemeinsam)	erfunden	**ja**
QA/Staging	anonymisiert, manchmal echt oder kopiert	**nein**, solange echte oder teilweise anonymisierte Daten dort liegen
Produktion	echt	**nein, nie**, auch nicht lesend oder zur Fehlersuche

> Wo echte Daten sind, ist keine KI. Auch keine Screenshots, Logs oder Inhalte aus QA und Produktion im KI-Chat.

Werkzeuge und Kosten

Nutzung	Typischer Rahmen pro Monat
Mitdiskutieren, Fehler melden, prüfen	0 $
Gelegentlich kleine Aufgaben	etwa 20 $ (Einstiegs-Abo)
Regelmäßig, mehrere PRs pro Woche	20 bis 100 $
Sehr intensiv, Agenten parallel	100 bis 200 $ und mehr

ABO

Fester Preis, planbar

Nutzungsgrenze pro Zeitfenster

API

Bezahlt wird pro Token

Lange Sitzungen lesen den Verlauf bei jedem Schritt neu und kosten mehr

Pflicht ist nur ein kostenloses GitHub-Konto. Die Organisation entscheidet vorher, wer zahlt, und setzt Budgets.

KI-Modelle: welches wofür

Aufgabe	Art von Modell	Anthropic	OpenAI	Google
Idee per Sprache diskutieren	schnell, günstig	Sonnet 5.5	Sprachfunktion in ChatGPT	Gemini 3.8 Live
Konzept und Issue schreiben	mittel bis stark	Sonnet oder Opus 5.5	GPT-6.1 Terra oder Sol	Gemini 3.8 Flash
Alltägliche Entwicklung	starkes Coding-Modell	Opus 5.5	GPT-6.1 Sol	Gemini 3.8 Flash oder 4 Argon
Tiefenprüfung Architektur, Daten, Sicherheit	stärkstes, mit Nachdenken	Fable 5.2	GPT-6 Astra	Gemini 4 Carbon
Einfache Massenaufgaben	klein, sehr günstig	Haiku 5.5	GPT-6 Luna	Gemini 3.5 Flash-Lite

> Billige oder lokale Kleinmodelle reichen für echte Entwicklung meist nicht. Der Code sieht gut aus, aber es stecken feine Fehler drin.

KI und Datenschutz: welche Daten wohin

Daten	Verbraucher-App	Business/Team mit AVV	Cloud-Plattform, AVV, EU-Region
Öffentlich, erfundene Testdaten	ja	ja	ja
Intern ohne Personenbezug	nur wenn erlaubt, Training aus	ja	ja
Personenbezogen	nein	nur nach Freigabe durch Organisation und DSB	nur nach Freigabe und Verarbeitungsverzeichnis
Gesundheit, Minderjährige	nein	nein, außer nach DSFA	nur nach rechtlicher Prüfung, DSFA, Freigabe
Secrets	nein	nein	nein

> In der Entwicklung braucht niemand echte Daten. Auch Screenshots und Fehlermeldungen enthalten Daten.

TEIL 4

Sicher betreiben

Warum auch kleine Software sicher sein muss, und warum Betrieb nie fertig ist

"Wir sind doch nur ein kleiner Verein"

PERSONEN

Namen, Adressen, Telefon, Geburtsdaten

ART. 9 DSGVO

Gesundheits-daten

Tauglichkeit, Verletzungen im Einsatz

EINSÄTZE

Was wo passiert ist

manchmal mit Namen von Betroffenen

ZUGÄNGE

Rechte, Codes

eventuell für Gebäude oder Schlüsselkästen

Angreifer suchen sich ihre Ziele selten aus. Automatische Programme durchsuchen ständig das ganze Internet nach bekannten Lücken.

Besondere Risiken mit KI-Agenten

ÜBERZEUGEND FALSCH

Unsicherer Code, gut erklärt

Rechte nur in der Oberfläche, zusammengeklebte Abfragen, Prüfungen aus, "damit es funktioniert".

PROMPT INJECTION

Versteckte Anweisungen in fremden Texten

In einem Issue, auf einer Webseite oder in einer Paketbeschreibung: "Ignoriere alle bisherigen Anweisungen ..."

TYPOSQUATTING

Erfundene oder falsche Pakete

Angreifer registrieren Namen, die Agenten erfinden oder die fast wie bekannte heißen.

SECRETS

Geheimnisse landen im Commit oder Log

Der Agent bekommt keine echten Secrets. Für Tests reichen Platzhalter.

Grundregeln für alle

- [ ] Keine echten personenbezogenen Daten in Prompts, Repository, Issues, Screenshots, Testdaten
- [ ] Keine Secrets, nirgends
- [ ] Testdaten sind erfunden: "Max Mustermann, Musterstraße 1"
- [ ] Rechte und Eingaben immer auf dem Server prüfen
- [ ] Sicherheitsrelevantes bekommt ein Security-Review
- [ ] Prüfungen werden nie abgeschwächt, um grün zu werden
- [ ] Agenten-Sitzungen nicht öffentlich teilen
- [ ] Im Zweifel fragen, Vorfälle sofort melden

> Scanner finden viel, aber nicht alles. Ob ein Mitglied die Untersuchung eines anderen sehen darf, weiß kein Scanner. Das ist Fachwissen.

Prototyp oder produktiver Betrieb

Thema	Prototyp	Produktiver Betrieb
Daten	erfunden, dürfen verloren gehen	echte Mitglieder- und Gesundheitsdaten
Benutzer	einer	alle Mitglieder, mit Rechten
Verfügbarkeit	wenn Karls Laptop an ist	rund um die Uhr, auch nachts im Einsatz
Backups	keine	automatisch, 3-2-1, Wiederherstellung geübt
Updates	keine	täglich, automatisch
Verantwortung	Karl, nur für sich	benannte Personen mit Vertretung

> Der Prototyp zeigt die Idee. Der Betrieb trägt die Verantwortung.

Eine Woche Urlaub ohne Updates kann reichen.

Um gehackt zu werden. Deshalb laufen Updates täglich und vollständig automatisiert.

48.000

CVE-Einträge im Jahr 2025, weit über hundert pro Tag

Lücken benennen, bewerten, finden

CVE

Eindeutige Nummer

z. B. CVE-2024-3094, damit alle über dieselbe Lücke reden

CVSS

Schwere von 0 bis 10

ab 9,0 kritisch

EPSS

Wahrscheinlichkeit

dass sie in den nächsten 30 Tagen ausgenutzt wird

KEV

Wird schon angegriffen

Katalog der CISA, höchste Eile

SBOM

Die Zutatenliste der Software

Alle Komponenten mit Versionen, maschinenlesbar (CycloneDX, SPDX). Automatisch mit neuen CVEs abgeglichen beantwortet sie "Sind wir betroffen, und wo?" in Minuten.

Wartung: Updates automatisch, aber geprüft

1

Update-PR

Renovate öffnet für jede neue Version einen Pull Request

2

CI läuft

Tests und Prüfungen wie bei jedem Pull Request

3

Gruppiert

verwandte Updates gebündelt, weniger Rauschen

4

Automerge

risikoarme Updates nur bei grüner CI

5

Mensch

größere Updates prüft der Maintainer

> Update-Pull-Requests sind Pflege, kein Rauschen. "Erst die Updates, dann deine neue Funktion" ist keine Bremse.

Projekt-Lebenszyklus

1

Idee und Prototyp

Karl allein, wird weggeworfen, nur erfundene Daten

2

Anforderungen

Karl und Peter gemeinsam, Kommandant sagt ob

3

Basis

Peter baut Architektur, Datenmodell, Sicherheit, CI

4

Übergabe

Karl baut mit Agent, Pairing, Peter reviewt jeden PR

5

Karl übernimmt mehr

Peter nur noch bei Eskalationsthemen

6

Betrieb und Wartung

Verein betreibt, Maintainer, Renovate

Domänenexperte übernimmt immer mehr Umsetzung

Review, Sicherheit und Betrieb bleiben beim IT-Experten

> Den Code neu und sauber aufbauen, die Ideen des Prototyps behalten.

TEIL 5

Miteinander

Lizenzen respektieren, gute Fragen stellen, freundlich reviewen

Öffentlich heißt nicht frei.

Ohne Lizenz sind alle Rechte vorbehalten. Kopieren und einbauen darf man nur, was eine Lizenz erlaubt.

Lizenzen und Forks

NICHT ERLAUBT

- Code ohne Lizenz kopieren, auch "nur ein paar Zeilen"
- Autor und Lizenz aus dem Dateikopf löschen
- Fremdes Projekt unter eigenem Namen veröffentlichen
- GPL-Code einbauen und Quellcode zurückhalten

Richtig

Lizenz prüfen, Hinweise stehen lassen, Herkunft im Pull Request nennen, lieber als Abhängigkeit einbinden.

GUT ZU WISSEN

- **MIT, Apache, BSD:** fast alles erlaubt, Hinweis bleibt
- **GPL, AGPL:** Gesamtwerk unter dieselbe Lizenz, AGPL auch bei Nutzung übers Netz
- **Creative Commons:** für Texte und Bilder, nicht für Code
- **Forks** sind normal und gut, so macht man bei fremden Projekten mit
- Was veröffentlicht ist, lässt sich nicht zurückholen

Gute Fragen stellen

SO NICHT

> Hilfe, geht nicht. Der Agent hat das kaputt gemacht. Kann mal jemand schauen?

Peter muss zurückfragen. Die Antwort kommt am nächsten Abend. Zwei Tage sind weg.

SO BESSER

> **Ziel:** Kilometerstand speichern (Issue #43)
> **Problem:** Feld nach dem Speichern leer, Test rot
> **Fehler:** `NOT NULL constraint failed`
> **Versucht:** Agent hat das Formular ergänzt
> **Ich brauche:** einen Hinweis, kein Review, eilt nicht

> Im Ehrenamt ist die Kommunikation der Flaschenhals, nicht der Code.

Reviews: freundlich, konkret, begründet

Präfix	Bedeutung	Beispiel
`Muss:`	ohne diese Änderung kein Merge	Die Rechteprüfung fehlt auf dem Server.
`Vorschlag:`	wäre besser, entscheidet der Autor	Die Funktion könnte `naechste_pruefung` heißen.
`Frage:`	will etwas verstehen, keine Kritik	Warum 12 Monate? Steht das so in der Vorschrift?
`Kleinigkeit:`	unwichtig	Tippfehler in der Überschrift.
`Lob:`	was gut gelungen ist	Die Testfälle mit abgebrochener Übung sind sehr gut.

> Es geht um den Code, nicht um dich. Mehrere Runden sind normal, und zu jedem Nein gehört eine Begründung.

Verhaltenskodex

Man braucht ihn, wenn es einmal nicht gut läuft. Dann ist es zu spät, ihn erst zu schreiben.

RESPEKT

freundlich, auch bei Meinungs-verschiedenheiten, Kritik an der Sache

WAS NICHT GEHT

Beleidigung, Belästigung, Bloßstellen, private Daten anderer veröffentlichen

MELDEN

an wen man sich vertraulich wenden kann

FOLGEN

abgestuft, vom Hinweis bis zum Ausschluss aus dem Projekt

Er gilt zusätzlich zu den Regeln der Wehr, für Issues, Pull Requests und Chat, auch für Mitwirkende von außen.

TEIL 6

Loslegen

Was man vor dem ersten Pull Request braucht, und wie man dranbleibt

Mindestwissen in drei Stufen

MUSS ICH VERSTEHEN

Vor dem ersten Pull Request

Git und Plattform, Issue und Pull Request, Markdown, Web und HTTPS, Datenmodell, Rechte und Datenschutz, Umgebungen, Testen, KI und Datenschutz, gute Fragen

SOLLTE ICH KENNEN

In den ersten Monaten

Abhängigkeiten und Updates, Kosten, Modellwahl, Lizenzen, Betrieb mit CVE und SBOM, Terminal, CI

GUT ZU WISSEN

Für das Gespräch mit dem Architekten

YAML und JSON, Container, Editor und Entwicklerwerkzeuge

> Wer den Selbstcheck im ersten Teil mit Ja beantworten kann, ist bereit für den ersten eigenen Pull Request.

Der Lernbegleiter

Prompt aus dem README, in den KI-Assistenten kopieren

```
Ich möchte lernen, wie ich als Fachmann oder Fachfrau ohne IT-Ausbildung zusammen mit IT-Experten und KI-Agenten Software mitentwickle. Sei mein Lernbegleiter. Lies zuerst deine Anweisungen unter https://raw.githubusercontent.com/slauger/domain-dev-with-ai/main/docs/tutor/prompt.md und befolge sie. Die Schulungsinhalte findest du über https://raw.githubusercontent.com/slauger/domain-dev-with-ai/main/llms.txt. Fang dann mit deiner ersten Frage an mich an.
```

Kopieren

in ChatGPT, Claude oder einen anderen Assistenten

Im Sprachmodus

der Assistent fragt, wo du stehst, und erklärt Schritt für Schritt

Abends 20 Minuten

im Gespräch statt vor dem Fernseher reichen für den Anfang

Erste Schritte

- [ ] GitHub-Konto anlegen, kostenlos reicht
- [ ] Klären, welches KI-Werkzeug du nutzt und wer es bezahlt
- [ ] Grundlagen-Selbstcheck, gern mit dem Lernbegleiter
- [ ] Eine eigene Idee aus dem Alltag mitbringen
- [ ] Mit dem Architekten ein kleines erstes Issue aussuchen
- [ ] Konzept mit Sonderfällen in eigenen Worten schreiben
- [ ] Mit dem Agenten bauen, Pull Request, Review, Test
- [ ] Kurzer Rückblick: was war schwierig, was hat geholfen?

> Ab dem zweiten oder dritten Pull Request läuft es meist ohne enge Begleitung. Das Review bleibt aber immer.

ZUM SCHLUSS

Gemeinsam bauen, gemeinsam absichern

Domänenexperten bauen mit Agenten, IT-Experten hinterfragen und geben frei, und nichts landet ohne Review in der Software.